

ML Backend Lead: Personal Execution Plan

Task-specific responsibilities, daily milestones, APIs, schemas, tests, and handoff requirements

Internal build window
July 21-31, 2026

Submission deadline
Friday, July 31 at 11:59 PM ET

Working title: CreatorLens - an AI creative director for short-form video.

Your job is not to invent every possible machine-learning feature. Your job is to make the application produce reliable, evidence-grounded, structured creative feedback that the rest of the team can integrate and demonstrate.

1. Your Mission

- Own the backend contract from uploaded media evidence to AI-generated recommendations.
- Keep the AI output grounded in measured timestamps, transcript segments, and media features.
- Deliver stable JSON schemas early so the frontend and media pipeline can work independently.
- Integrate IBM Granite or watsonx in the judged flow and document exactly where it is used.
- Protect the team from scope explosion by choosing simple, testable analysis features.
- Support the final demo with deterministic test media and a reliable fallback path.

2. Personal Deliverables

Deliverable	Minimum acceptance criterion
Analysis request schema	Contains project goal, platform, audience, tone, target length, transcript segments, objective media findings, and video metadata.
Analysis response schema	Contains overall summary, category scores, timestamped feedback, prioritized edits, revised script, and automatic-fix eligibility.
Backend API	Frontend can upload a file, start analysis, poll status or await completion, retrieve results, and request an approved edit.
Evidence aggregation	Media/transcript outputs are normalized into one model-ready JSON object with timestamps in seconds.
IBM model integration	One controlled prompt produces parseable output; validation and retry logic handle malformed responses.
Grounding safeguards	Each feedback item references evidence and does not invent timestamps outside video duration.
Tests and fixtures	At least three sample payloads, schema validation tests, and one end-to-end happy-path test.
Documentation	Architecture notes, environment variables, model name/service, local run commands, and Bob usage examples.

3. Recommended API Contract

Method and route	Purpose	Your ownership
POST /api/projects	Create project and save creative brief.	Define request model and response ID.
POST /api/projects/{id}/media	Upload video and start preprocessing.	Coordinate file contract with media lead.
POST /api/projects/{id}/analyze	Run evidence aggregation and AI analysis.	Primary owner.
GET /api/projects/{id}/status	Return queued/processing/completed/failed and progress message.	Define stable states and errors.
GET /api/projects/{id}/analysis	Return validated structured feedback.	Primary owner.
POST /api/projects/{id}/edits	Submit approved edit IDs and render options.	Define request contract; media lead implements rendering.
GET /api/projects/{id}/preview	Return revised media URL or processing status.	Coordinate response contract.

4. Core Data Schema

Keep all times as floating-point seconds. Assign stable IDs to transcript segments, findings, feedback items, and recommended edits so the frontend can connect markers and user approvals.

```
analysis_input = {
  "project": {"goal": str, "platform": str, "audience": str,
  "tone": str, "target_length_seconds": number},
```

```
"video": {"duration_seconds": number, "fps": number,
"width": int, "height": int},
"transcript_segments": [
{"id": str, "start": number, "end": number, "text": str}
],
"findings": [
{"id": str, "category": str, "start": number, "end": number,
"metric": str, "value": any, "interpretation": str}
]
}

analysis_output = {
"summary": str,
"scores": {"story": int, "pacing": int, "audio": int,
"visuals": int, "accessibility": int},
"timeline_feedback": [
{"id": str, "start": number, "end": number,
"category": str, "severity": str, "evidence_ids": [str],
"explanation": str, "suggestion": str,
"automatic_fix": str | null}
],
"recommended_edits": [...],
"revised_script": str
}
```

5. Your Daily Plan

Date	Your highest-priority tasks	Personal exit criterion
Jul 21	Lock AI scope. Define categories, input/output schemas, endpoint list, model-access test, environment variables, and stored fixtures. Ask Bob to critique architecture and identify risks.	Frontend and media leads receive versioned example JSON. One IBM model call succeeds or access blocker is escalated.
Jul 22	Build FastAPI skeleton, Pydantic models, project state handling, mock analysis endpoint, and error response format. Add unit tests for schemas.	Frontend can call your backend and render a complete mock response.
Jul 23	Integrate media/transcript results into normalized evidence. Implement timestamp validation, finding IDs, simple derived features, and fixture generation.	One real video produces valid analysis_input.json with no manual editing.
Jul 24	Write system/task prompt, connect Granite/watsonx, parse structured output, validate, retry once, and fall back to a safe partial response when parsing fails.	Three test inputs produce schema-valid responses and no feedback timestamp exceeds video duration.
Jul 25	Expose live analysis to frontend. Fix latency, CORS, serialization, polling/status, and malformed request errors. Add result caching.	Upload-to-feedback works through the browser twice in a row.
Jul 26	Define recommended edit objects and automatic-fix mapping. Hand approved edit payloads to media lead. Add revised-script output and simple prioritization.	At least one AI recommendation maps to a real edit operation.
Jul 27	Integration freeze. Add end-to-end tests, sample video, deterministic fixture, logs, timeout handling, and clean install instructions.	Fresh clone can run the backend and complete the official demo flow.
Jul 28	Improve grounding, clarity, severity ranking, and score calibration. Remove vague or unsupported feedback. Test quiet audio, no speech, short video, and API failure.	Feedback is useful, evidence-based, and degrades gracefully on edge cases.
Jul 29	Write AI approach/architecture and IBM technology sections for README. Create architecture diagram inputs. Document Bob contributions and link journal entries.	README accurately explains the AI pipeline and can be reviewed without speaking to you.
Jul 30	Support demo recording, precompute backup analysis, verify production credentials, and prepare 20-second architecture explanation.	Both live and backup demo paths work.
Jul 31	Final smoke test, inspect public repo for secrets, verify model references and links, then freeze code.	No last-minute feature additions; judged flow is stable.

6. Technical Task Checklist

Backend foundation

- ☐ Create virtual environment and pinned requirements.
- ☐ Add configuration class and .env.example.
- ☐ Add structured logging and clear error messages.
- ☐ Set upload size/type limits.
- ☐ Use project-relative storage paths only.
- ☐ Add CORS configuration for the frontend origin.

Evidence processing

- ☐ Normalize transcript segments.
- ☐ Validate start <= end <= duration.

- [] Convert media metrics into human-readable findings.
- [] Generate stable IDs.
- [] Keep raw values separate from model interpretations.
- [] Store fixtures for development without reprocessing video.

Model layer

- [] Use a system prompt that forbids invented evidence.
- [] Request JSON only.
- [] Validate with Pydantic or JSON Schema.
- [] Retry malformed output once with validation feedback.
- [] Clamp or reject invalid scores and timestamps.
- [] Record model/service name and latency.
- [] Do not log secrets or full private user media.

Testing

- [] Schema unit tests.
- [] Timestamp bounds tests.
- [] Empty/no-speech input test.
- [] Malformed model response test.
- [] IBM API timeout test.
- [] End-to-end official demo fixture.
- [] Fresh-clone run test.

7. Prompt Requirements

- State the creator goal and target platform before evidence.
- Tell the model to behave as a collaborative creative director, not an absolute judge.
- Require every timestamped feedback item to cite one or more evidence IDs.
- Prohibit invented scenes, words, metrics, or timestamps.
- Separate objective evidence from subjective recommendation.
- Prioritize no more than five high-impact edits for the MVP.
- Mark an automatic fix only when the backend actually supports that operation.
- Return concise text suitable for a UI card.

8. Acceptance Tests for the Official Demo

Test	Pass condition
Upload and analyze	A supported video produces a completed analysis without manual backend intervention.
Timestamp navigation	Clicking feedback jumps within one second of the intended segment.
Grounding	Every high-severity item is supported by transcript or measured media evidence.
Model failure	The app returns a useful error or fallback instead of crashing.

Test	Pass condition
Automatic edit	At least one recommendation can be approved and rendered.
Reproducibility	The official sample works after restarting services or on a fresh clone.
Public safety	No API key, local path, private media, or oversized generated artifact is committed.

9. Personal Scope Guardrails

Do not spend the remaining time training a custom deep-learning model. The winning technical story is a disciplined multimodal pipeline that measures media, grounds an IBM model, returns structured recommendations, and applies one recommendation. Reliability beats novelty theater.

- Prefer deterministic signal-processing features over unverified semantic claims.
- Use sampled frames rather than trying to process every frame with a vision model.
- Do not own the frontend; provide fixtures and contracts, then let the frontend lead build.
- Do not own all FFmpeg work; define the edit contract and help integration.
- Stop prompt polishing once the official demo produces clear, supported feedback consistently.